EPFL - Computer Science

COM-407 Advanced Networks

Lecture 06 — October 16, 2025

Author: Jonathan Dutoit

Professor: Pavlos Nikoupoulos

Transport Layer (Part 1)

TCP and UDP

(MAC Continued) 6. Security Aspects

- MAC addresses are sent in cleartext
- Many packets are broadcast (especially in shared-medium LANs)

Possible attacks:

- Eavesdropping
- Free riding
- Impersonation

Solutions

- Access control: requires uses to show credentials before allowing access to the network;
  - shared = user knows a password, same for all (e.g., WiFI WPA-Personal)
  - per-user = user has a personal password (e.g., WiFi WPA-Enterprise
  - **prevents free riding**
- Authenticity: we can use cryptography to authenticate src MAC address
  - **prevents impersonation**, easier if access control is per use
- Confidentiality: we can use cryptography to encrypt MAC frame payloads (but not MAC addresses)
  - **prevents eavesdropping**

At home:

- WiFi WPA-Personal provides access control but does not authenticate individual users
- Impersonation is possible by whoever knows the shared WiFI password

EPFL:

- WiFi uses per-user access control, authentication and encryption
- Eavesdropping, Free-riding and impersonation are no longer possible

Securing the MAC layer

IEEE 802.1X for access control

IEEE 802.1AE (MACSEC) for authentication and encryption

steps:
i. Host negotiates access with WiFI base station (BS) or Ethernet switch
ii. BS/Switch contacts an authentication server (e.g., RADIUS)
iii. Host and server establish a secure (TLS) connection and run an inner authentication protocol (e.g., PEAP/MSCHAPv2), where host "shows" credentials and session keys are commonly derived
iv. If host is accepted, frames between host and BS/Switch are authenticated and encrypted using session keys

- authentication covers MAC payload and src MAC address
- encryption covers MAC payload

Conclusion

- The MAC layer fro wireless medium (WiFI)
  takes care of **sharing** the radio waves in a local environment
- The MAC layer for wired medium (Ethernet)

- 3

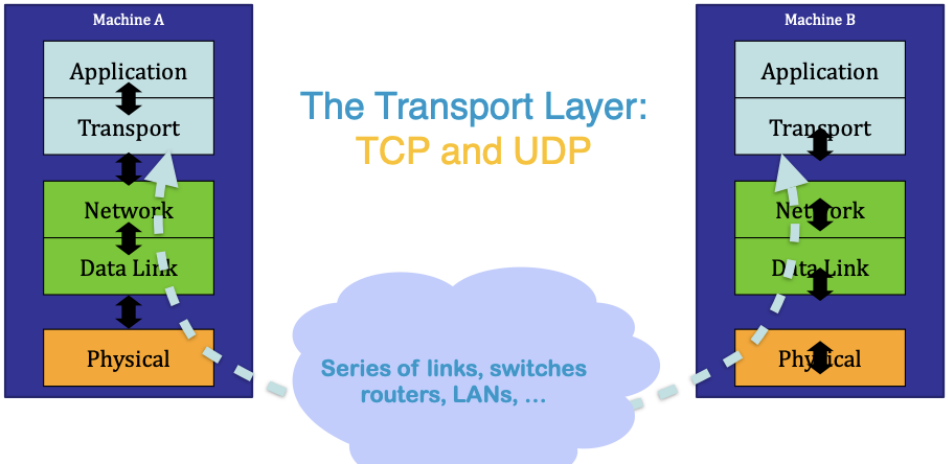
We saw 2 mechanisms to interconnect devices:

  - the network layer (with IP router), and
  - the MAC layer (with switches)

What are the differences?

- Switches are plug-and-play devices, independent from IP numbering plans and whether IPv4 or IPv6 is used (as opposed to router).
- but use forwarding tables that are not structured (no aggregation by prefix).
  - For every received frame, a switch must search the entire table, because of the exact match.
  - If network becomes too large, table size / lookup times may become impractical.
    * Prefer router for large networks
- Historically, routers were in software and switches in hardware; nowadays, every combination exists.

Transport Layer



Contents

1. UDP
2. TCP Basics: Sliding Window and Flow Control
3. TCP Connections and Sockets
4. More TCP Bells and Whistles
5. Secure Transport
6. Where should packet losses be repaired ?

Reminder from 1st lecture: Transport layer services

- Network + Data link + Physical layer carry packets end-toend
- packets may be lost due to:
  - errors at the physical layer
  - buffer-overflow events at router-s/switches
- or reordered due to:
  - following different paths to destination

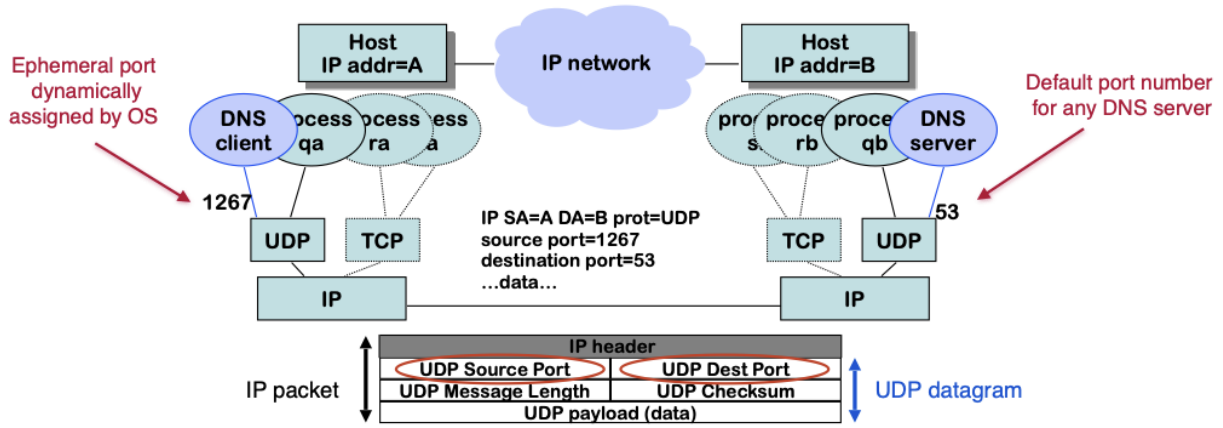
- Transport layer
  - makes networks services available to pgraoms
  - is in end-systems only, not in routers' data plane (i.e. not at forwarding level)
  - may handle packet loss/reordering or not
    - UDP not reliable transfer, takes no action
    - TCP reliable, in-order transfer by using sophisticated mechanisms

What is the definition of a "server" in networks?

A. A machine that hosts resources to be used in the web
B. A computer with high CPU performance
C. A computer with large data storage
D. THe role of a program

Reminder from 1st lecture: Port Numbers

- Assigned by OS to identify processes within a host
- servers' port numbers must be well-known to clients (e.g., 53 for DNS, 80 for HTTP, 443 for HTTPS)
- src and dest port numbers are inside transport-layer header



1. UDP: message-oriented and unreliable

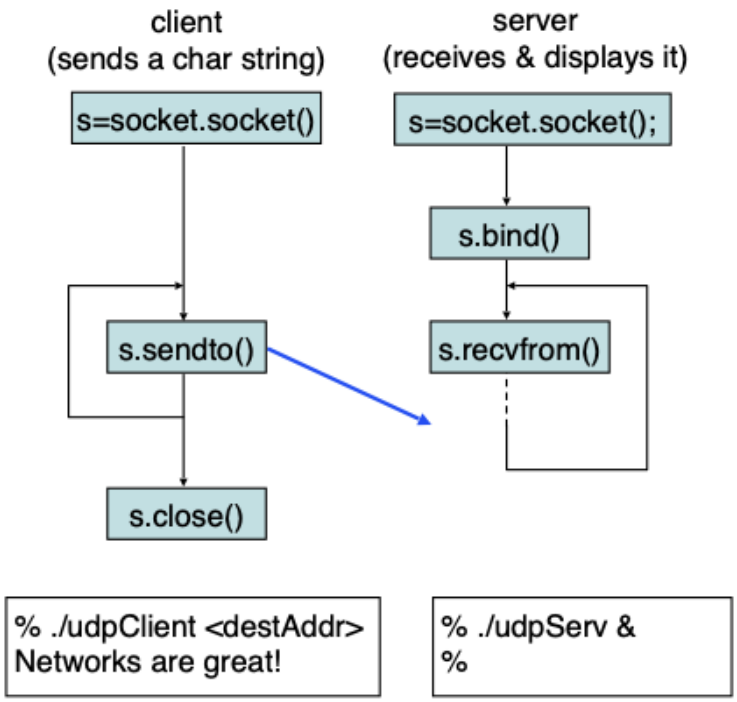
- UDP delivers either the exact message (a.k.a. "datagram") or nothing
- Each messages $\leq$ 65535 bytes (32 bits sequences)
- If a message is too large to fit into a single IP packet (i.e. larger than MTU), then IP layer fragments it:
  - not visible to the transport layer
  - at source's IP layer, metadata about fragments is added to IP header
  - if a fragment/piece is lost, then the entire message is lost
- Application layer must handle out of order or lost messages, if needed
  - e.g., DNS client resends DNS query if no response is received

How is UDP implement in practice?

Via sockets = programming interfaces

- (in Unix) for read/write: sockets ≈ files

- `socket(socket.AF_INET, ...)` resp. `socket(socket.AF_INET6, ...)` creates and IPv4 (resp. IPv6) socket, returns an ID (= file descriptor)
- bind() associates an IP address + port number with a socket, can be skipped for the client
  - Port = 0 *Rightarrow* any available port
  - IP address = 0 ⇒ all host's IP addresses
- sendto() specifies: dest IP addresses, dest port number, and the message to send
- recvfrom() blocks until a message is received for this port number; it returns: src IP address, src port number and the message.



What socket to open? IPv4 or IPv6?

- Transport layer is not affected by the choice of IP (no UDPv6, not TCPv6)
- But, an application must choose IPv4 or IPv6, or better support both

How? Use `socker.getaddrinfo()` to let the DNS give you whatever is available

IPv6 sockets can be dual-stack

= bound to both IPv6 and IPv4 addresses of the local host

= receive packets from IPv6 and/or IPv4 correspondents

How? The correspondents' IPv4 addresses are mapped to IPv6 addresses

  - using the IPv4-mapped IPv5 address format
  - i.e., by appending the IPv4 address to prefix `::ffff:0:0/96`

- Default in Linux, must be enabled for every socket (with `setsockopt` in Windows.
- 2
- 3

How does the Operating System view UDP?

Note: buffers can overflow! Receiver buffer is needed to assemble messages, sender buffer is necessary for congestion (for example WiFI carrier waits the channel to be free)

Recap - UDP

On the sending side: OS sends the UDP messages ASAP, but also uses a buffer to store data if interface is busy. OS may algo fragment the message if needed.

On the receiving side: OS re-assembles IP fragments of UDP message (if needed) and keeps the message in received buffer ready to be read. The message is:

2. TCP offers reliable, in-order delivery

What?

- TCP guarantees that all data is delivered in order and without loss, unless the connection is broken

How?

- Detects reordering and loss, via sophisticated mechanisms:
  - per-byte sequences numbers
  - a connection-setup phase for the sender/receiver to synchronize their sequence numbs
  - acknowledgements; if loss is detected, TCP retransmits
- further optimizations, e.g.:
  - flow control avoids buffer at the receiver
  - TCP knows the allowable maximum segment size (MSS) and segments data accordingly ⇒ avoids fragmentation at the IP layer